

Jev in the Wild: A Data-Driven Analysis of the Jev Model’s Functionality, Applications and Ecosystem

Guoming Ling¹, Muen Xue¹, Zijian Ye²

¹Sun Yat-sen University, ²The Chinese University of Hong Kong

Abstract

Jev is a fast, low-cost decision model that answers natural-language questions with choices, binary judgments, and scores. As its public ecosystem grows rapidly, it remains unclear how Jev is used across applications and how public attention relates to project distribution. To answer these questions, we conduct a large-scale, data-driven analysis of 2,170 publicly available Jev projects collected from GitHub as of September 22, 2026. We find rapid early growth in Jev’s public ecosystem, with both new projects and integration into existing repositories. Across diverse domains, projects use Jev for multiple decision purposes and combine its interfaces. Attribute judgment and scoring are widely used, while the use of action selection, content filtering, and model and tool selection varies across domains. These patterns suggest that Jev serves as a reusable decision component whose functionality varies with the surrounding workflow. Meanwhile, public attention is concentrated in routing and interface agents and does not track project counts. Our findings provide a quantitative view of Jev’s emerging ecosystem and inform the design and evaluation of general-purpose decision models across diverse application contexts.

1 Introduction

Jev is a recently introduced model for fast, low-cost decisions defined through natural language (TypeSafe AI, 2026). Developers specify a decision question and its possible outputs, and Jev returns a choice, probability, or score. It provides three interfaces: *Choice* selects among candidate options, *Noul* makes binary judgments, and *Score* rates an input against predefined levels (Almeida, 2026). Together, these interfaces support lightweight decisions such as model routing, context filtering, and action selection (Li et al., 2026b; Jiang et al., 2026).

Why is Jev useful? Many applications require frequent, lightweight decisions (Mozzarelli and

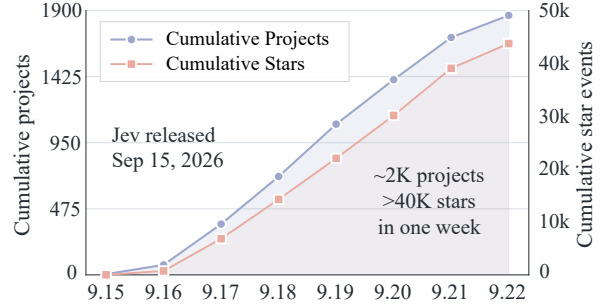


Figure 1: **Jev’s ecosystem grew explosively** in its first week after release on September 15, 2026, expanding to 1,865 new GitHub repositories and gaining 43,750 stars. This rapid rise demonstrates strong early momentum.

Schneider, 2026), but existing models do not fully meet this need. Traditional supervised classifiers require task-specific labeled data and training, and changes to the label space often require collecting new data and retraining the model (Devlin et al., 2019). LLMs provide much stronger zero-shot generalization and avoid task-specific training (Brown et al., 2020), but their autoregressive generation introduces substantial latency and inference cost (Kim et al., 2023), especially for simple decisions with a small output space. Jev targets this gap by combining zero-shot generalization with fast and low-cost inference (Ibrahim and Zaki, 2026). This makes it suitable for applications that require many small, diverse, and frequently changing decisions.

How is Jev used in practice? Jev is typically used as a lightweight decision component within a larger system (Jiang et al., 2026). The surrounding application prepares the relevant state and defines the decision, while Jev evaluates the request and returns the result (Li et al., 2026b). The system then uses this result to control the next operation (Wu and Lim, 2026). In this way, Jev handles the decision itself, while the surrounding system manages task-specific context and execution.

As shown in Figure 1, Jev has rapidly grown

Major category	Subcategory	Projects	% of total	Stars	% of stars
Interface Agents	Web Interaction	125	5.8%	46,067	21.0%
	Desktop & Mobile	50	2.3%	1,418	0.6%
Software Engineering	Code Quality	90	4.1%	1,500	0.7%
	Dev Workflows	218	10.0%	3,014	1.4%
Search & Memory	Context & Memory	77	3.5%	10,533	4.8%
	Search & Data	120	5.5%	2,334	1.1%
	Classification	184	8.5%	1,169	0.5%
Safety & Governance	Review & Approval	82	3.8%	5,750	2.6%
	Security & Compliance	93	4.3%	1,904	0.9%
Routing & Automation	Model & Tool Routing	132	6.1%	71,671	32.6%
	Workflow Automation	118	5.4%	19,332	8.8%
Simulation & Control	Games & Simulation	227	10.5%	1,825	0.8%
	Robotics & Control	25	1.2%	264	0.1%
Content & Expert Tasks	Professional Tasks	176	8.1%	6,597	3.0%
	Content & Dialogue	211	9.7%	5,363	2.4%
Infrastructure & Other	SDKs & Tools	175	8.1%	34,800	15.8%
	Research & Resources	56	2.6%	6,010	2.7%
	Other Applications	11	0.5%	107	< 0.1%

Table 1: Distribution of 2,170 public Jev projects across 8 major categories and 18 subcategories. Public Jev projects span a broad range of application domains, showing that **Jev is already being used for diverse decision tasks**.

from a new model into an emerging ecosystem of public projects. However, it remains unclear how this ecosystem is evolving and how Jev is actually used across applications. To answer these questions, we conduct a large-scale, data-driven analysis of 2,170 publicly available Jev projects collected from GitHub as of September 22, 2026. We systematically examine the growth of the ecosystem, the applications and decision roles of Jev, and how public attention is distributed across these applications. Our analysis provides a quantitative view of the emerging Jev ecosystem and a reference for understanding how the model is being adopted in practice. We discuss related work in Appendix A and summarize our contribution:

- **Growth trends.** We quantify the growth of public Jev projects and GitHub attention over time, revealing rapid and bursty ecosystem growth.
- **Usage patterns.** We characterize how Jev is used through application domains, decision purposes, and interface adoption, and compare project distribution with public attention.

2 Data Collection and Growth Trends

In this section, we first describe the dataset and then analyze project growth and community attention.

2.1 Data Collection

We construct the dataset in three stages: candidate retrieval, project verification, and annotation.

Candidate retrieval. We collect candidate repositories from GitHub using predefined search queries based on Jev-related keywords, API endpoints, and SDK references. We combine repository and code search and deduplicate the retrieved repositories using GitHub repository IDs.

Project verification. Each candidate repository is inspected by a GPT-6 Luna Max agent. We include a repository only when its public code or documentation provides clear evidence that Jev is used for a concrete task. Repositories that only mention Jev or provide generic wrappers or SDKs without a concrete application are excluded.

Annotation. For each included project, a GPT-6 Luna Max agent annotates its primary application

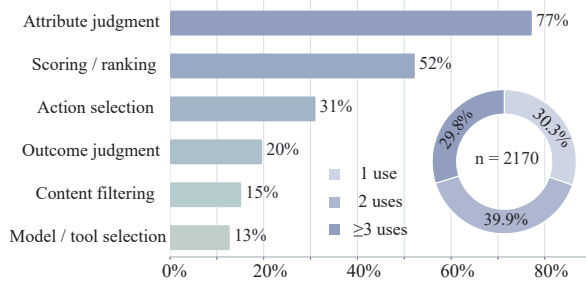


Figure 2: The bars show the share of projects for each decision purpose, and the ring shows how many purposes are identified per project. Attribute judgment is the most common purpose, while most projects with an identified purpose use Jev for more than one.

domain, decision purposes, interface usage, and other attributes required by our analyses. Application domain describes what the project is built for, while decision purpose describes the role performed by Jev within the project. Key inclusion decisions and application-domain labels are independently reviewed by a second GPT-6 Luna Max agent, and disagreements are resolved by inspecting the original repository materials. This process yields 2,170 verified Jev projects for our analysis.

2.2 Growth Trends

Figure 1 shows that Jev was adopted rapidly after its release on September 15, 2026. Of the 2,170 repositories in our dataset, 1,865 were created in the following week, while 305 existing repositories integrated Jev. This pattern shows that Jev attracted both new development and adoption by established projects. Public attention increased just as quickly. During the same week, these new repositories gained 43,750 stars, rapidly drawing public attention. Growth in both repositories and stars continued beyond this initial surge, indicating strong and sustained momentum across the Jev ecosystem.

3 Jev Usage Patterns

We characterize Jev’s use across domains, decision purposes, and interfaces, then examine how project activity relates to public attention.

3.1 Application Domains

Table 1 shows that Jev is used across a broad range of application domains. The two largest categories, Content & Expert Tasks and Search & Memory, account for only 17.8% and 17.6% of projects, respectively, showing that no single application dominates the ecosystem. In contrast, public attention

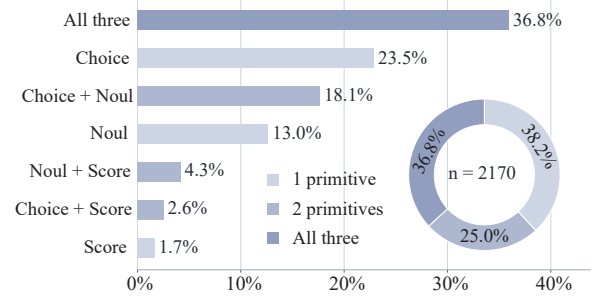


Figure 3: The bars show the share of projects using each combination of Choice, Noul, and Score. The ring shows how many of these interfaces are identified per project. **All three interfaces are widely used.**

is more concentrated. Routing & Automation receives 41.4% of all stars, with Model & Tool Routing alone accounting for 32.6%. Overall, Jev is being explored across diverse decision tasks, while community attention is particularly concentrated on routing and system integration.

3.2 Decision Purposes and Interface Adoption

We examine what Jev decides and which interfaces projects use. A project can have several purposes and interfaces, so individual shares may overlap.

Decision purposes. Figure 2 summarizes six decision purposes. Each bar reports the share of projects associated with a given purpose, with each project counted once per purpose, while the ring indicates the number of purposes identified in each project. Attribute judgment is the most common, appearing in 77% of projects, followed by scoring or ranking in 52% and action selection in 31%. Among projects with an identified purpose, 69.7% use Jev for two or more purposes, indicating that Jev often supports multiple types of decisions within a single application.

Interface adoption. Figure 3 compares how projects combine Jev’s three interfaces. The bars give the share of projects using each exact combination of Choice, Noul, and Score. The ring shows whether a project uses one, two, or all three interfaces. Choice appears in 81.0% of projects, Noul in 72.2%, and Score in 45.4%. Most projects use at least two interfaces, and 36.8% use all three, showing that all three interfaces are widely used. Similar decision models should therefore support categorical selection, binary judgment, and scoring to facilitate developer use.

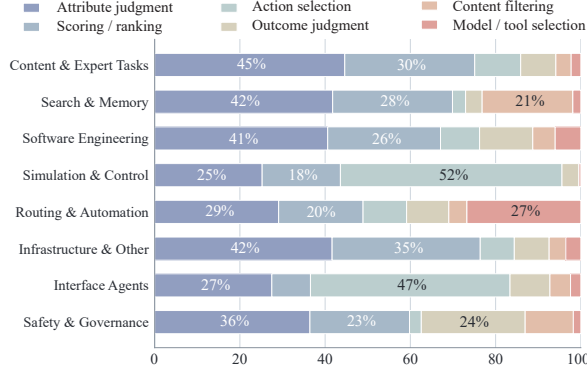


Figure 4: Decision purposes across project categories. Each bar shows the share of observed purpose labels within a category. The changing mix shows how **the surrounding workflow shapes Jev’s functionality**, from judging inputs to choosing actions and tools.

3.3 Purpose Composition Across Domains

Figure 4 shows that Jev’s role varies across project categories. Action selection accounts for 52% of purpose labels in Simulation & Control and 47% in Interface Agents, reflecting Jev’s use in guiding moves and interactions. Model and tool selection accounts for 27% in Routing & Automation but only 6% in Software Engineering. Content filtering constitutes 21% of labels in Search & Memory, whereas outcome judgment accounts for 24% in Safety & Governance. Despite these differences, attribute judgment remains the leading purpose in six of the eight categories. Together, attribute judgment and scoring account for about 70% of labels in Search & Memory and 75% in Content & Expert Tasks. Overall, Jev serves as a reusable judgment component whose role adapts to application inputs and downstream actions.

3.4 Project Supply and Public Attention

Figure 5 contrasts project supply with public attention across application categories. We use project count as supply and mean GitHub stars per project as a proxy for demand. Because the measures are normalized independently, their distance from the center indicates how each category compares with the corresponding reference value. Public attention is far more concentrated than project activity. Routing & Automation contains 250 projects and averages 364 stars per project, while Interface Agents contains 175 and averages 271. Together, they represent only 19.6% of projects but receive 63.0% of all stars. By contrast, Content & Expert Tasks and Search & Memory are the largest categories, with 387 and 381 projects, but average just 31 and

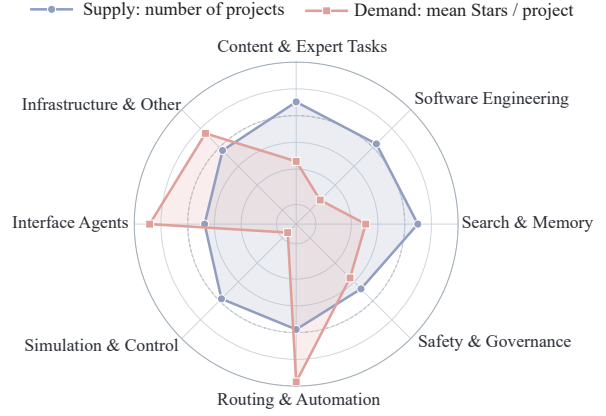


Figure 5: Project supply and public attention across eight categories. Supply is the number of projects, and demand is proxied by mean GitHub Stars per project. The two series are divided by their respective reference values of 271 projects per category and 101 Stars per project, then plotted on a logarithmic radial scale. Routing & Automation and Interface Agents receive disproportionate attention relative to their project counts.

37 stars per project. The disparity is especially clear between Routing & Automation and Simulation & Control. Despite nearly identical project counts of 250 and 252, they average 364 and 8 stars per project. Thus, public attention does not track project volume. Because stars measure interest in entire repositories and may be concentrated in a few prominent projects, these results indicate uneven visibility rather than unmet demand.

4 Conclusion

In this paper, we present a large-scale data-driven analysis of Jev’s early public ecosystem, revealing rapid growth, diverse applications, and a mismatch between project distribution and public attention. Across domains, projects commonly combine multiple decision purposes and interfaces, while surrounding applications determine how model outputs guide subsequent actions. These findings suggest that general-purpose decision models should support flexible combinations of selection, judgment, and scoring. Their evaluation should cover both recurring decision purposes and the application contexts in which they are used. The concentration of public attention further motivates including less visible applications when constructing representative evaluations. Our dataset and taxonomy provide a basis for selecting these settings and studying how decision models are integrated into larger systems. Overall, this work provides a foundation for future decision-model research.

Limitations

First, our analysis provides a snapshot of the Jev ecosystem as of September 22, 2026. As the ecosystem evolves, project counts, usage patterns, and public attention may shift, which could change the conclusions drawn here. Second, our dataset includes only public GitHub repositories. Private repositories and commercial applications are outside the scope of our analysis and may exhibit different usage patterns. Our findings therefore do not provide a complete account of Jev adoption.

References

- Diogo Almeida. 2026. [Introducing system one models & jev](#). TypeSafe AI Blog. Published September 15, 2026. Accessed: September 24, 2026.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. [Language models are few-shot learners](#). In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.
- browser-use. 2026. [Jev ultra-fast](#). GitHub repository, commit 1231850a0bf1a0c0341fe408ef1668dbbdfdac46. Accessed: September 24, 2026.
- Zehua Cheng, Wei Dai, and Jiahao Sun. 2026. [this-that-model-1.0: A typed decision model that decides in 30 ms, for a millionth of a cent](#). *Preprint*, arXiv:2609.23886.
- Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *Journal of machine learning research*, 25(70):1–53.
- Christopher Clarke, Yuzhao Heng, Yiping Kang, Krisztian Flautner, Lingjia Tang, and Jason Mars. 2023. Label agnostic pre-training for zero-shot text classification. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 1009–1021.
- Boyuan Deng, Shuyi Fan, Hongyang Zhang, and Xinhong Xie. 2026. [Jev for scientific decisions: Evaluating semantic choices and their consequences](#). *Preprint*, arXiv:2609.24965.
- devagrawal09. 2026. [Jev review](#). GitHub repository, commit 31f89602797fb7bea007f8a480bf368bf564954e. Accessed: September 24, 2026.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [BERT: Pre-training of deep bidirectional transformers for language understanding](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.
- fhshaik. 2026. [Typesafe mario](#). GitHub repository, commit ca22449ed187118d19326d1f54b01b6636578aa4. Accessed: September 24, 2026.
- gargpratyush. 2026. [jev-router](#). GitHub repository, commit 38da6b84ea01241bfc41fbddc0928d0f40a703f0. Accessed: September 24, 2026.
- Kishaloy Halder, Alan Akbik, Josip Krapac, and Roland Vollgraf. 2020. Task-aware representation of sentences for generic text classification. In *Proceedings of the 28th international conference on computational linguistics*, pages 3202–3213.
- Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2020. Deberta: Decoding-enhanced bert with disentangled attention. *arXiv preprint arXiv:2006.03654*.
- Hazem Ibrahim and Yasir Zaki. 2026. [Evaluating decision models for text annotation in computational social science](#). *Preprint*, arXiv:2609.24574.
- Dongming Jiang, Yi Li, and Bingzhe Li. 2026. [Jev-mem: System-one-controlled agentic memory for efficient ai agents](#). *Preprint*, arXiv:2609.23986.
- Sehoon Kim, Karttikeya Mangalam, Suhong Moon, Jitendra Malik, Michael Mahoney, Amir Gholami, and Kurt Keutzer. 2023. [Speculative decoding with big little decoder](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 39236–39256. Curran Associates, Inc.
- kyotofin. 2026. [tax-doc-classifier](#). GitHub repository, commit 3e95a77f763c6becb78472f8b2ce2f54237f9214. Accessed: September 24, 2026.
- Moritz Laurer, Wouter van Atteveldt, Andreu Casas, and Kasper Welbers. 2023. Building efficient universal classifiers with natural language inference. *arXiv preprint arXiv:2312.17543*.
- Delong Li, Xu Wang, Haochen Gong, Rui Lang, and Guangsheng Yu. 2026a. [Fast intent-driven service orchestration with jev for 6g edge networks](#). *Preprint*, arXiv:2609.23136.
- Delong Li, Xu Wang, Haochen Gong, Rui Lang, and Guangsheng Yu. 2026b. [Replacing large language models with jev decision models for low-latency edge service orchestration](#). *Preprint*, arXiv:2609.22753.
- Yubo Li, Yidi Miao, Ramayya Krishnan, and Rema Padman. 2026c. [Jev-as-a-judge: Accept when confident, escalate when unsure](#). *Preprint*, arXiv:2609.26550.

- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Michele Mozzarelli and Giulia Schneider. 2026. Ai and strategic decisions: Facing the incompleteness. *European Business Organization Law Review*, pages 1–33.
- Raul Puri and Bryan Catanzaro. 2019. Zero-shot text classification with generative language models. *arXiv preprint arXiv:1912.10165*.
- Amir Rafe and Subasish Das. 2026. [Calibrated decisions at scale: Converting police crash narratives into probabilistic crash variables with a system one model \(jev\)](#). *Preprint*, arXiv:2609.24052.
- Werner Robitza. 2026. [Jevqa - video quality from metadata, bitstream, and pixel features with a general-purpose decision model](#). *Preprint*, arXiv:2609.24395.
- Victor Sanh, Albert Webson, Colin Raffel, Stephen H Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Teven Le Scao, Arun Raja, et al. 2021. Multitask prompted training enables zero-shot task generalization. *arXiv preprint arXiv:2110.08207*.
- Ihor Stepanov, Mykhailo Shtopko, Dmytro Vodianytskyi, Oleksandr Lukashov, Alexander Yavorskyi, and Mykyta Yaroshenko. 2025. Gliclass: Generalist lightweight model for sequence classification tasks. *arXiv preprint arXiv:2508.07662*.
- Yu Sun and Junhao Xu. 2026. [Type-safe is not error-free: A constrained decision head follows the option name, not the rubric bound to it](#). *Preprint*, arXiv:2609.26758.
- tamaratran. 2026. [Fast jev compaction](#). GitHub repository, commit e3f262a7f4d42bd8dd32ced30d26176f7cb545b0. Accessed: September 24, 2026.
- TypeSafe AI. 2026. Typesafe api. <https://api.typesafe.ai/docs>. Version 0.2.0. Accessed: September 24, 2026.
- Junjie Wang, Ping Yang, Ruyi Gan, Yuxiang Zhang, Jiaying Zhang, and Tetsuya Sakai. 2023. Zero-shot learners for natural language understanding via a unified multiple-choice perspective. *IEEE Access*, 11:142829–142845.
- Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, Atharva Naik, Arjun Ashok, Arut Selvan Dhanasekaran, Anjana Arunkumar, David Stap, et al. 2022. Super-naturalinstructions: Generalization via declarative instructions on 1600+ nlp tasks. In *Proceedings of the 2022 conference on empirical methods in natural language processing*, pages 5085–5109.
- Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V Le. 2022. [Finetuned language models are zero-shot learners](#). In *International Conference on Learning Representations*.
- Tiantong Wu and Wei Yang Bryan Lim. 2026. [Reflex with jev for efficient selective control in llm agents](#). *Preprint*, arXiv:2609.26532.
- Haike Xu, Zongyu Lin, Jing Zhou, Yanan Zheng, and Zhilin Yang. 2023. A universal discriminator for zero-shot generalization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10559–10575.
- y0usaf. 2026. [pi-jev](#). GitHub repository, commit 80a80f32ad281f3d905429edfe0e5b8d7977d82a. Accessed: September 24, 2026.
- Wenpeng Yin, Jamaal Hay, and Dan Roth. 2019. Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 3914–3923.
- Guanxu Yu and Yuhang Yao. 2026. [Visual jev: Accurate and efficient decisions from shared visual context](#). *Preprint*, arXiv:2609.25845.
- Urchade Zaratiana, Gil Pasternak, Oliver Boyd, George Hurn-Maloney, and Ash Lewis. 2025. Gliner2: An efficient multi-task information extraction system with schema-driven interface. *arXiv preprint arXiv:2507.18546*.
- Urchade Zaratiana, Nadi Tomeh, Pierre Holat, and Thierry Charnois. 2024. Gliner: Generalist model for named entity recognition using bidirectional transformer. In *Proceedings of the 2024 conference of the north american chapter of the association for computational linguistics: Human language technologies (volume 1: Long papers)*, pages 5364–5376.
- Ming Zhong, Yang Liu, Da Yin, Yuning Mao, Yizhu Jiao, Pengfei Liu, Chenguang Zhu, Heng Ji, and Jiawei Han. 2022. Towards a unified multi-dimensional evaluator for text generation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 2023–2038.

A Related Work

We review the development of classification methods, existing research on Jev applications and evaluation, and closely related decision models.

A.1 Supervised and Zero-Shot Classification

Supervised text classifiers typically fine-tune pre-trained encoders on task-specific labeled data to predict predefined categories (Devlin et al., 2019; Liu et al., 2019; He et al., 2020). Zero-shot methods reduce this dependence by using natural-language label descriptions to transfer across categories and tasks (Yin et al., 2019; Halder et al., 2020; Clarke et al., 2023). Generative language models extend this flexibility through task descriptions and examples supplied in prompts, without parameter updates for each target task (Brown et al., 2020; Puri and Catanzaro, 2019). Instruction tuning improves generalization by training on diverse tasks expressed in natural language (Wei et al., 2022; Sanh et al., 2021), with later work expanding task coverage and model scale (Wang et al., 2022; Chung et al., 2024). In practice, supervised classifiers often require additional labeled data and retraining as tasks or label sets change (Halder et al., 2020). LLMs avoid task-specific training, but autoregressive generation can incur substantial latency and inference cost (Brown et al., 2020). Jev targets this gap by combining zero-shot generalization with fast, low-cost inference for frequent and diverse decisions (Almeida, 2026; Ibrahim and Zaki, 2026). Our work analyzes public Jev projects to examine how the model is used across applications and how its ecosystem is developing.

A.2 Jev Applications and Evaluation

Existing studies examine Jev’s use within applications and its performance on decision tasks. Service orchestration studies use Jev to translate natural-language requests into structured requirements for scheduling (Li et al., 2026b,a). Agent systems apply Jev to memory organization and retrieval control in Jev-Mem (Jiang et al., 2026), and to action selection with confidence-based fallback to a stronger LLM in REFLEX (Wu and Lim, 2026). Other applications include annotating crash narratives (Rafe and Das, 2026), selecting semantic relations before scientific calculations (Deng et al., 2026), and estimating video quality from metadata and extracted features (Robitza, 2026). Evaluations of text annotation and model judging assess

accuracy, confidence calibration, and inference cost (Ibrahim and Zaki, 2026; Li et al., 2026c). Their results show that accuracy and calibration vary across tasks, while routing uncertain cases to stronger LLMs can improve the balance between accuracy and cost. Robustness tests also reveal sensitivity to option names, showing that valid output types alone do not ensure correct decisions (Sun and Xu, 2026). Our work complements these evaluations of particular tasks and systems with a systematic data-driven analysis of Jev’s public ecosystem.

A.3 General-Purpose Decision Models

Related models combine flexible task definitions with efficient, structured predictions. Universal classifiers use natural language inference to score label descriptions (Laurer et al., 2023), while UniMC and the Universal Discriminator select or score candidate answers across tasks (Wang et al., 2023; Xu et al., 2023). Compact encoders support a similar use of label descriptions: GLiNER identifies entities of user-specified types (Zaratiana et al., 2024), and GLiClass jointly encodes text and candidate labels for classification (Stepanov et al., 2025). GLiNER2 combines classification and extraction through a shared schema (Zaratiana et al., 2025), while UniEval expresses text evaluation criteria as binary questions (Zhong et al., 2022). To reduce repeated computation, `this-that-model-1.0` returns probabilities for multiple questions in one forward pass (Cheng et al., 2026), while the Jev-inspired Visual Jev shares visual context across independent questions (Yu and Yao, 2026). Jev shares these goals through its `Choice`, `Noul`, and `Score` interfaces for software integration (Almeida, 2026). These studies focus on model design and prediction quality. Our work complements them by examining how Jev is integrated into applications across its public ecosystem.

B Representative Applications

These examples show how different applications turn Jev’s typed decisions into program actions. The diagrams summarize source code paths rather than recorded executions.

B.1 Jev Ultrafast: Browser Actions

As shown in Fig. 6, Jev Ultrafast is a browser-agent implementation that uses Jev to select actions over a structured representation of the current webpage. It illustrates how Jev can serve as a lightweight decision layer in an interactive agent loop.

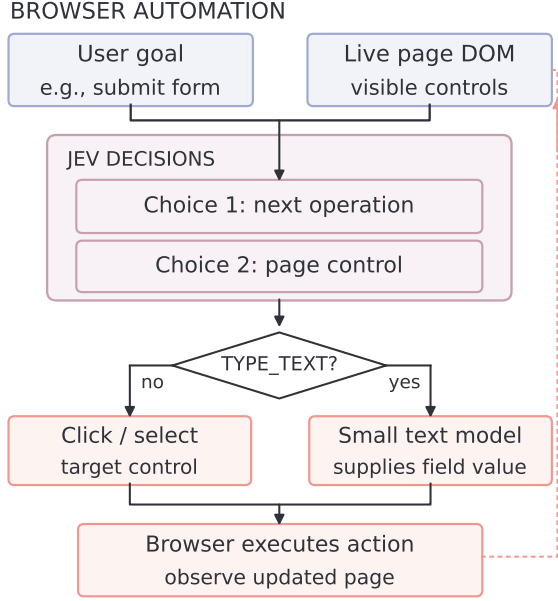


Figure 6: Jev selects a browser operation and target from the live page state. Text for typing comes from a separate model.

Task and Input. Jev Ultrafast drives a browser toward a user-specified goal using a structured representation of the current page state (browser-use, 2026). At each step, it sends Jev the goal, recent actions, page context, and an indexed list of interactive controls with their labels, current values, and supported operations. For example, a search box is represented as an editable target rather than as pixels in a screenshot.

Jev Decision. In a single request, the system submits a `Choice` question for the next operation and separate target questions for the available operation types. Jev may choose an operation such as clicking, typing, selecting an option, scrolling, waiting, or terminating the task. The agent then uses only the target answer associated with the selected operation, so that the selected target is compatible with that operation. Jev selects the action and target but does not generate the text to be entered into a field.

Use of the Decision. After validating the returned choice, the browser executor applies the selected action to the corresponding live DOM element. When the selected operation is typing, a separate small language model generates the field value from the goal and page context. This division places Jev in the role of action selector, while the browser executor performs interaction and the text model handles language generation.

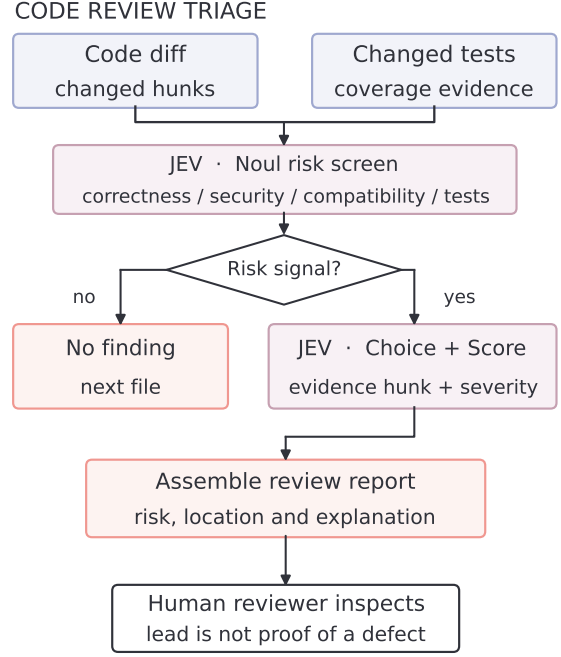


Figure 7: Risk judgments and evidence selection turn a code diff into review leads for a human.

B.2 Jev Review: Code Review Triage

As shown in Fig. 7, Jev Review applies Jev to code-review triage, helping identify which changes deserve closer inspection and where supporting evidence can be found. It demonstrates the use of Jev for structured risk screening and prioritization.

Task and Input. Jev Review inspects a code change to help a human reviewer focus attention (devagrawal09, 2026). The system provides Jev with structured information extracted from the pull request, including changed files, individual patch hunks, and modified or newly added tests. These inputs allow the workflow to reason about whether a potential concern is actually supported by the submitted diff and, when relevant, identify the specific changed region that deserves closer inspection.

Jev Decision. Several `Noul` questions screen for correctness, security, reliability, compatibility, and test-coverage concerns. When a risk signal is detected, additional `Choice` questions select a supporting diff hunk and classify a possible failure mechanism. A `Score` question then estimates the priority or severity of the concern. The resulting outputs are structured judgments—for example, a risk signal, supporting code location, failure category, and severity score—rather than a free-form code review.

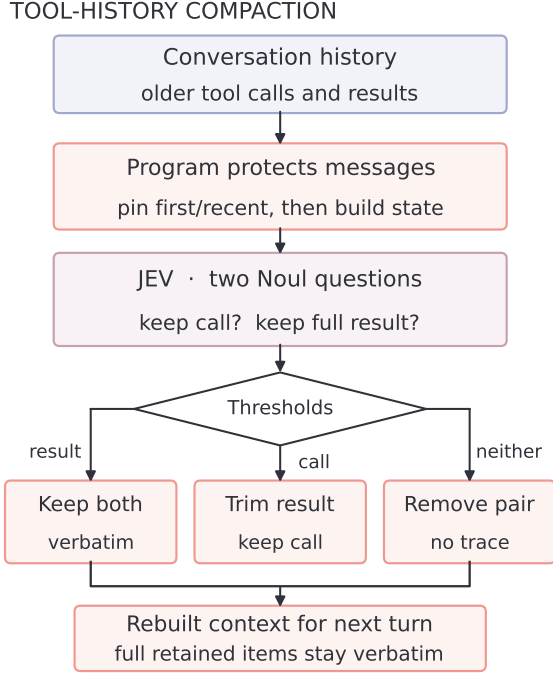


Figure 8: Two keep judgments guide whether an older tool exchange is retained, shortened, or removed.

Use of the Decision. The surrounding program combines these structured outputs into a review report that highlights suspicious changes and directs the reviewer to the relevant evidence. Findings can therefore be ranked and inspected without requiring Jev to generate a complete natural-language review. Importantly, each reported concern remains a prompt for further inspection rather than proof of a defect: the final assessment is left to the human reviewer, together with tests, static analysis, and other validation tools. This example illustrates how Jev can serve as a triage layer that narrows the search space and prioritizes review effort within a larger software-engineering workflow.

B.3 Fast Jev Compaction: Tool History

As shown in Fig. 8, Fast Jev Compaction uses Jev to reduce the tool history retained in long-running agent conversations. Rather than summarizing previous interactions, it decides which tool calls and results remain useful for future turns.

Task and Input. Fast Jev Compaction reduces the amount of tool history carried into later agent turns (tamaratran, 2026). Instead of rewriting the conversation, it presents Jev with a compact representation of the current context and evaluates eligible older tool interactions individually. Each

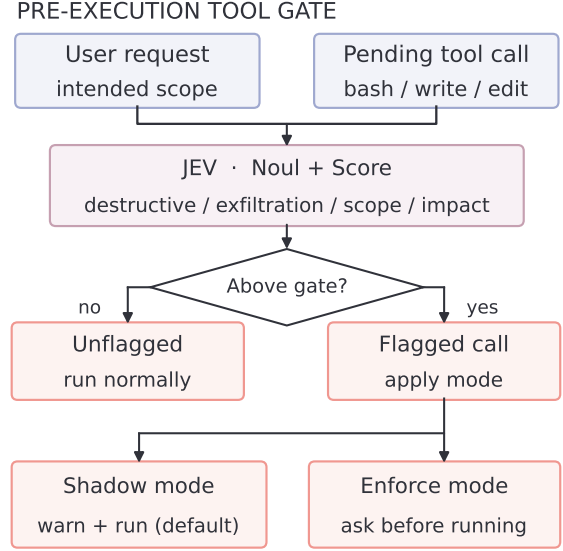


Figure 9: A typed risk assessment informs a tool gate. The extension mode controls warning or confirmation.

interaction consists of a tool call and its corresponding result. Recent messages and explicitly pinned content are protected from removal, while ordinary user and assistant messages are not subject to this Jev-based filtering step.

Jev Decision. For each eligible tool interaction, two `Noul` questions ask whether remembering the tool call remains useful for continuing the task and whether the full tool result is still needed. Jev returns a keep probability for each question. The surrounding program compares these probabilities against configured thresholds, turning the model outputs into retention decisions. Importantly, Jev is not asked to summarize or rewrite the historical content; it only estimates whether different parts of the interaction should remain available.

Use of the Decision. The resulting decisions lead to three possible outcomes. If both the call and its result remain useful, the complete interaction is retained. If the call is useful but the full result is not, the call is preserved while the result is shortened. If neither is considered useful, the call–result pair is removed from the carried context. Content that survives this process remains verbatim except for the explicit truncation of selected results. This example illustrates Jev acting as a selective context-retention mechanism: it determines which parts of an agent’s tool history should survive without generating a replacement summary.

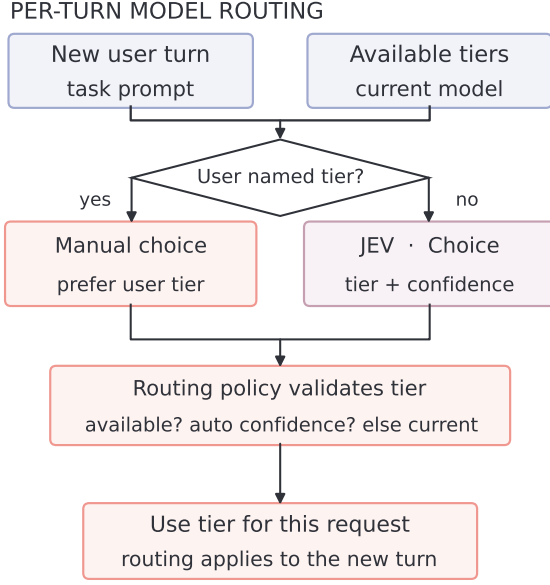


Figure 10: Jev proposes a model tier for a new turn. Routing policy applies the choice or a user override.

B.4 pi-jev: Tool-Call Risk Gate

As shown in Fig. 9, Pi-jev places Jev in front of agent tool execution as a lightweight risk gate. It evaluates whether a proposed action may be destructive, expose data, or exceed the user’s requested scope before the tool is executed.

Task and Input. pi-jev checks a pending agent tool call before execution (yOusaf, 2026). It sends Jev the user request, the tool name, and relevant arguments for operations such as shell commands or file edits. The intended task provides context for judging whether an operation goes beyond what the user asked.

Jev Decision. Noul questions assess destructive behavior, data exfiltration, and whether the operation exceeds the requested scope. A Score question rates potential impact. The program compares the returned probabilities and score with its configured thresholds to decide whether the call warrants attention.

Use of the Decision. In the default shadow mode, a flagged call produces a warning and can still run. In enforce mode, an interactive session asks the user to confirm before it runs. Thus Jev supplies a risk signal, while the extension’s mode and policy determine whether that signal triggers a user confirmation.

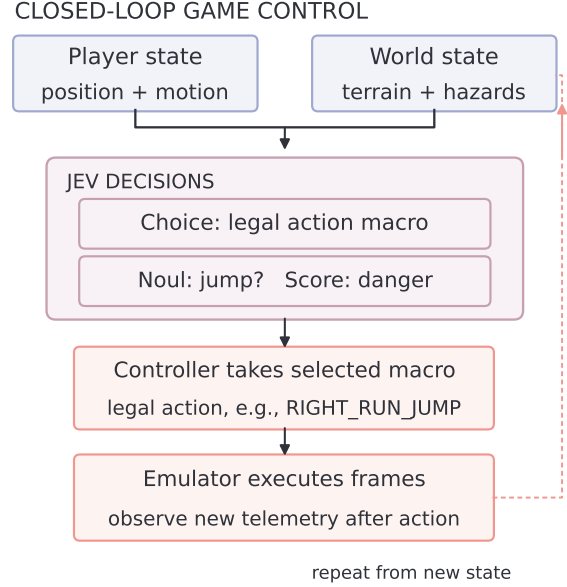


Figure 11: Structured telemetry leads to a legal action, which the emulator executes before the next observation.

B.5 jev-router: Per-Turn Model Selection

As shown in Fig. 10, Jev-router uses Jev to select an appropriate model tier for each new user request. It demonstrates how Jev can support dynamic model routing while leaving the final routing policy to deterministic code.

Task and Input. jev-router chooses a model tier for a new assistant request (gargpratyush, 2026). It considers the user’s current task and the model tiers available to the account. The routing policy also knows the model already in use and checks whether the user explicitly named a preferred tier.

Jev Decision. For an automatically routed turn, Jev recommends a model tier together with a confidence value. Its judgment describes which available class of model fits the request. The code validates the answer against supported tiers before switching models.

Use of the Decision. The policy applies the chosen tier to the current request, subject to availability and confidence rules. An explicit user choice takes precedence, and an invalid or missing Jev answer leaves the current tier in place. This example uses Jev for task-sensitive routing while keeping final control in deterministic policy code.

B.6 TypeSafe Mario: Game Control

As shown in Fig. 11, TypeSafe Mario uses Jev as part of a closed-loop controller for a game emulator. Given structured game state, Jev selects among a small set of legal actions that are then executed in the emulator.

Task and Input. TypeSafe Mario is an experimental controller for a game emulator (fhshaik, 2026). Instead of sending screenshots, it converts emulator telemetry and RAM into a compact structured state describing the player’s position and motion, jump trajectory, nearby enemies, terrain, recent control outcomes, and timing information. Jev also receives a small predefined set of legal controller macros from which it may choose.

Jev Decision. A `Choice` question selects the next legal controller macro. A `Noul` question estimates whether a forward jump is useful at the current moment, while a `Score` question rates immediate danger. These judgments are evaluated over the same game state, but the controller action itself is determined by the `Choice`. The selected macro is therefore a discrete state-to-action decision rather than a generated game plan.

Use of the Decision. The emulator executes the selected macro for several frames, records a new structured state, and repeats the cycle. Timing calculations and state extraction remain in deterministic code, while Jev interprets the resulting state and chooses among the available actions. Jev therefore acts as a state-to-action policy inside a closed feedback loop. The project demonstrates this control integration pattern rather than providing a general benchmark of game-playing performance.

B.7 tax-doc-classifier: Tax Form Pages

As shown in Fig. 12, Tax-doc-classifier uses Jev to classify individual pages of IRS tax documents. It treats Jev as a probabilistic classifier that predicts both the form identity and page type from extracted document text.

Task and Input. tax-doc-classifier identifies the IRS form and page kind of each PDF page (kyotofin, 2026). A PDF text extractor provides structured page text, while a generated registry supplies descriptions of IRS forms and schedules. Each page is classified independently, and blank pages can be detected without invoking Jev.

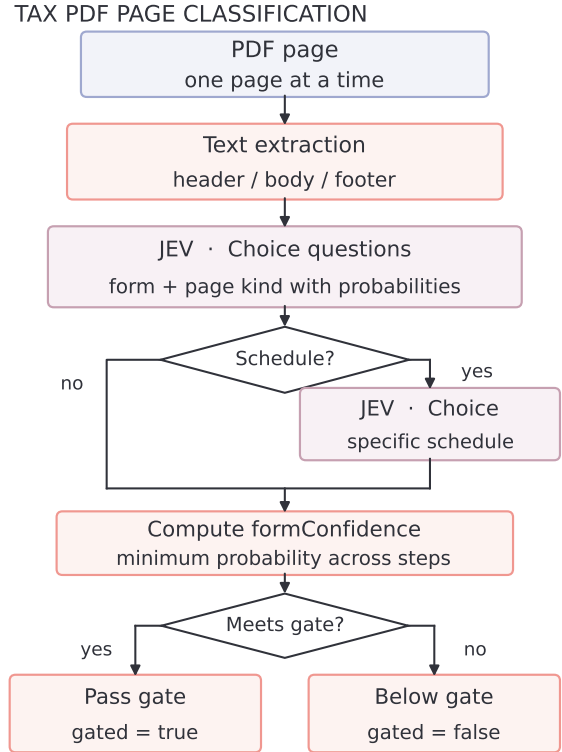


Figure 12: Page text is classified by Jev, then program code applies its own confidence gate.

Jev Decision. For a text-bearing page, Jev answers `Choice` questions over candidate IRS forms and seven page kinds, returning selected labels and option probabilities. Most pages are classified in one request. For several form families, a second narrower `Choice` distinguishes the parent form from its schedules. The resulting form confidence is the minimum confidence along the classification path, while the acceptance threshold is set by the surrounding program.

Use of the Decision. The program returns the predicted form, page kind, confidence, and whether the prediction passes a configurable gate. High-confidence results can be accepted for downstream processing, while lower-confidence cases can be deferred. Thus Jev provides probabilistic classification, while trust and fallback decisions remain in deterministic code.